

University of Asia Pacific

Department of Computer Science and Engineering

CSE 402: Operating System Lab

Lab Report: 05

Comparative Performance Analysis of Preemptive and Non-Preemptive Priority Scheduling

Submitted by:

Name: Faiaz Masnun Labib
Student ID: 23101128
Section: C-1

Submitted to:

Atia Rahman Orthi
Lecturer,
Department of CSE, UAP

Date of Submission: October 12, 2026

1. Problem Specification

- **Objective:** Implement, trace, and evaluate Preemptive and Non-Preemptive Priority CPU Scheduling.
- **Priority Convention:** Smaller numerical value signifies higher scheduling priority (0 = highest priority).
- **Evaluation Formulas:**
  - Completion Time (CT): Time at which process execution terminates.
  - Turnaround Time (TAT) = CT − AT
  - Waiting Time (WT) = TAT − BT
  - Average TAT = $\frac{1}{n}\sum_{i=1}^{n} \text{TAT}_i$, Average WT = $\frac{1}{n}\sum_{i=1}^{n} \text{WT}_i$

Process	Arrival Time (AT)	Burst Time (BT)	Priority	Relative Rank
P1	0 ms	8 ms	3	Lowest
P2	1 ms	2 ms	1	Second Highest
P3	2 ms	4 ms	2	Third Highest
P4	3 ms	1 ms	0	Highest

Table 1: Input Process Queue Parameters

2. Python Source Code

```
processes = [
    ["P1", 0, 8, 3],
    ["P2", 1, 2, 1],
    ["P3", 2, 4, 2],
    ["P4", 3, 1, 0]
]
n = len(processes)

# --- 1. Non-Preemptive Priority Scheduling ---
time, done, ct_non, sequence_non = 0, [], [0] * n, []
while len(done) < n:
    ready = [i for i in range(n) if i not in done and processes[i][1] <=
        time]
    if not ready:
        time += 1
        continue
    pick = min(ready, key=lambda i: processes[i][3])
    time += processes[pick][2]
    ct_non[pick] = time
    done.append(pick)
    sequence_non.append(processes[pick][0])

tat_non = [ct_non[i] - processes[i][1] for i in range(n)]
wt_non  = [tat_non[i] - processes[i][2] for i in range(n)]

# --- 2. Preemptive Priority Scheduling ---
```

```
rem = [processes[i][2] for i in range(n)]
time, ct_pre, sequence_pre = 0, [0] * n, []
while sum(rem) > 0:
    ready = [i for i in range(n) if processes[i][1] <= time and rem[i] > 0]
    if not ready:
        time += 1
        continue
    pick = min(ready, key=lambda i: processes[i][3])
    rem[pick] -= 1
    time += 1
    if not sequence_pre or sequence_pre[-1] != processes[pick][0]:
        sequence_pre.append(processes[pick][0])
    if rem[pick] == 0:
        ct_pre[pick] = time

tat_pre = [ct_pre[i] - processes[i][1] for i in range(n)]
wt_pre  = [tat_pre[i] - processes[i][2] for i in range(n)]

# --- 3. Output Generation ---
print("NON-PREEMPTIVE PRIORITY")
print("Execution Sequence: ", " -> ".join(sequence_non))
print("{:<8}{:<6}{:<6}{:<6}".format("PID", "CT", "TAT", "WT"))
for i in range(n):
    print("{:<8}{:<6}{:<6}{:<6}".format(processes[i][0], ct_non[i], tat_non
        [i], wt_non[i]))
print("Average TAT: {:.2f} ms | Average WT: {:.2f} ms\n".format(sum(tat_non
    )/n, sum(wt_non)/n))

print("PREEMPTIVE PRIORITY")
print("Execution Sequence: ", " -> ".join(sequence_pre))
print("{:<8}{:<6}{:<6}{:<6}".format("PID", "CT", "TAT", "WT"))
for i in range(n):
    print("{:<8}{:<6}{:<6}{:<6}".format(processes[i][0], ct_pre[i], tat_pre
        [i], wt_pre[i]))
print("Average TAT: {:.2f} ms | Average WT: {:.2f} ms\n".format(sum(tat_pre
    )/n, sum(wt_pre)/n))
```

GitHub Repository Link: Click Here

3. Step-by-Step Trace & Quantitative Results

A. Non-Preemptive Priority Trace

- $[0 - 8$ ms]: Only P1 is ready at $t = 0$. P1 executes uninterrupted until termination ($CT = 8$).
- $[8 - 9$ ms]: Ready queue contains $\{P2, P3, P4\}$. P4 has the highest priority (0). P4 executes to completion ($CT = 9$).
- $[9 - 11$ ms]: Ready queue contains $\{P2, P3\}$. P2 has priority 1. P2 executes to completion ($CT = 11$).
- $[11 - 15$ ms]: P3 executes to completion ($CT = 15$).
- **Execution Sequence:** P1 $\rightarrow$ P4 $\rightarrow$ P2 $\rightarrow$ P3

Process	Arrival Time	Burst Time	CT (ms)	TAT = CT - AT	WT = TAT - BT
P1	0 ms	8 ms	8	8 ms	0 ms
P2	1 ms	2 ms	11	10 ms	8 ms
P3	2 ms	4 ms	15	13 ms	9 ms
P4	3 ms	1 ms	9	6 ms	5 ms
Average				**9.25 ms**	**5.50 ms**

B. Preemptive Priority Trace

- $[0 - 1$ ms]: P1 begins execution ($Rem = 7$).
- $[1 - 3$ ms]: P2 arrives (Priority 1 vs 3) and preempts P1. P3 arrives at $t = 2$ (Priority 2), but P2 retains CPU. P2 completes ($CT = 3$).
- $[3 - 4$ ms]: P4 arrives (Priority 0) and gets CPU immediately. P4 completes ($CT = 4$).
- $[4 - 8$ ms]: Ready tasks are P3 (Priority 2, $Rem = 4$) and P1 (Priority 3, $Rem = 7$). P3 runs to completion ($CT = 8$).
- $[8 - 15$ ms]: P1 resumes for its remaining 7 ms and finishes ($CT = 15$).
- **Execution Sequence:** P1 $\rightarrow$ P2 $\rightarrow$ P4 $\rightarrow$ P3 $\rightarrow$ P1

Process	Arrival Time	Burst Time	CT (ms)	TAT = CT - AT	WT = TAT - BT
P1	0 ms	8 ms	15	15 ms	7 ms
P2	1 ms	2 ms	3	2 ms	0 ms
P3	2 ms	4 ms	8	6 ms	2 ms
P4	3 ms	1 ms	4	1 ms	0 ms
Average				**6.00 ms**	**2.25 ms**

C. Comparative Benchmark

Scheduling Method	Average Turnaround Time	Average Waiting Time	Status
Non-Preemptive	9.25 ms	5.50 ms	Inferior
Preemptive	6.00 ms	2.25 ms	**Superior**

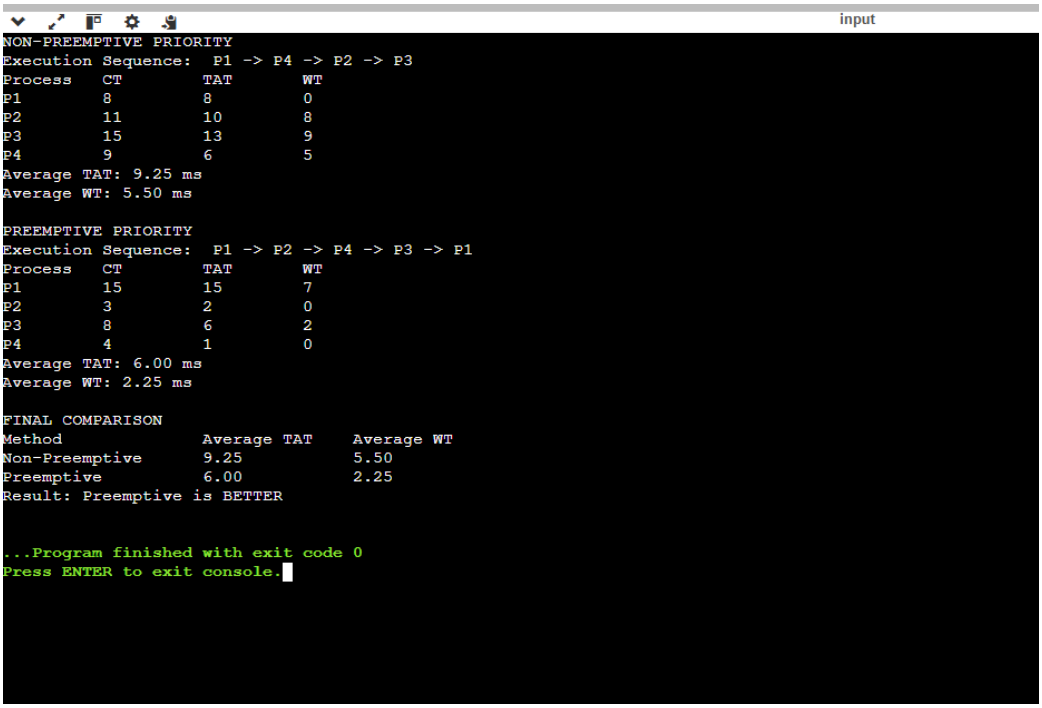


Figure 1: Console Output Verification

4. Comparative Evaluation: Which Approach Performs Better?

Decision: Preemptive Priority Scheduling performs significantly better than Non-Preemptive Priority Scheduling for this process configuration.

- **Direct Metric Improvement:**

  ΔAverage TAT = 9.25 ms − 6.00 ms = **3.25** ms reduction (35.1% faster)

  ΔAverage WT = 5.50 ms − 2.25 ms = **3.25** ms reduction (59.1% less queue delay)

- **Root Cause (Convoy Phenomenon in Non-Preemptive):**
  - In Non-Preemptive mode, low-priority process P1 holds the CPU for its full 8 ms burst.
  - Urgent, short burst tasks (P4 with $BT = 1$ ms, P2 with $BT = 2$ ms) arrive early but remain completely blocked until $t = 8$ ms. This severely degrades average responsiveness.

- **Preemption Advantage:**
  - Preemptive mode halts P1 immediately at $t = 1$ ms, clearing higher-priority tasks (P2, P4, P3) through the CPU rapidly.
  - P4 finishes in just 1 ms of turnaround time, and P2 completes in 2 ms, keeping queues short and maximizing throughput.

5. Discussion

- **Impact of Arrival Timings on Preemption:** Preemptive scheduling exhibits optimal behavior when lengthy, low-priority processes arrive first, followed immediately by short, high-priority tasks. Preemption eliminates head-of-line blocking.
- **CPU Burst vs. Context Switch Trade-Off:** While Preemptive scheduling introduces additional context switches (P1 is split across two execution slices: $0 \rightarrow 1$ ms and $8 \rightarrow 15$ ms), the dramatic 59.1% drop in average waiting time heavily justifies the minimal operating system switching overhead.
- **Final Takeaway:** For interactive systems where short, urgent system calls must execute with low latency, Preemptive Priority Scheduling is overwhelmingly preferred over Non-Preemptive scheduling.